Web Development Foundations

Week 8 – "Async Time Matters"

Day 1: "Async – Time Matters"

1

Async Is About Time

Async is about time, *not magic*.

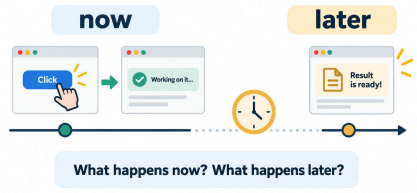
Some work:

- happens now
- starts now but finishes later
- needs the page to stay understandable while waiting

Today we ask:

- What happens now?
- What happens later?

2



Async Is About Time

3

Lab Review: Successful Path

Previous lab review:

- working behavior
- named functions
- clear responsibilities
- same result after refactoring

Success path:

Structure helps us see where timing belongs.



4



From Structure To Timing

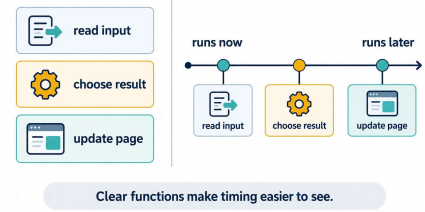
Last week:

- What does this function do?
- What is this function responsible for?

This week:

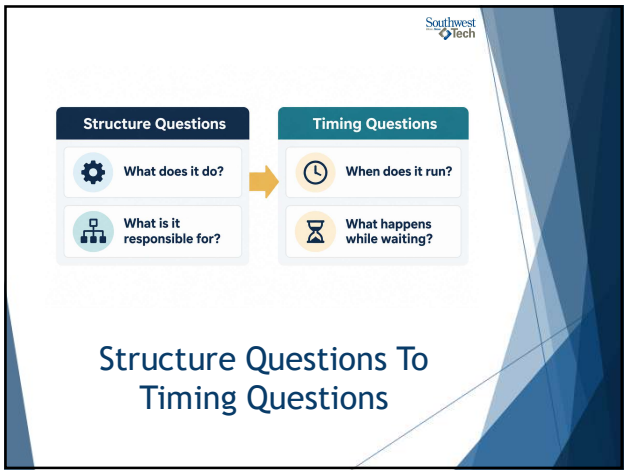
- When does this function run?
- What happens while we wait?

5

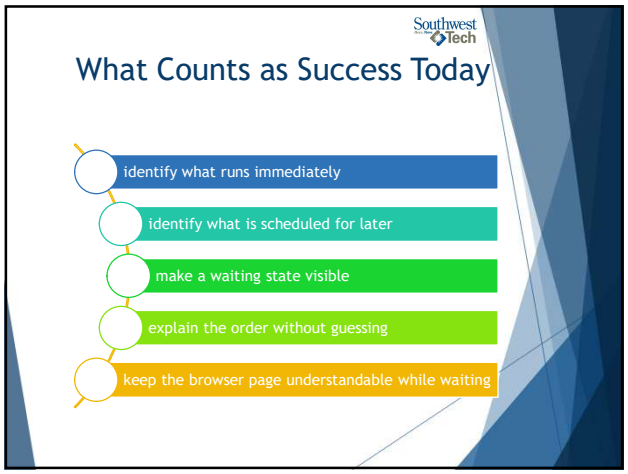


Week 7 Structure To Timing

6


Structure Questions
What does it do?
What is it responsible for?
Timing Questions
When does it run?
What happens while waiting?
Structure Questions To Timing Questions

7


What Counts as Success Today
identify what runs immediately
identify what is scheduled for later
make a waiting state visible
explain the order without guessing
keep the browser page understandable while waiting

8


What We Will Use Today
synchronous
asynchronous
delay
callback
`setTimeout()`
waiting state
visible feedback
retest

9


synchronous
asynchronous
delay
callback
setTimeout()
waiting state
visible feedback
retest
Async Toolbox

10


What We Will "Skip" For Now
Parked for later:
full Promise chains
advanced error handling
API authentication
CORS details
service workers
complex loading systems
For now:
Make timing visible.
Explain the order.

11


Parked For Later
Promise chains
error handling
authentication
CORS
service workers
complex loading systems
Today: visible timing.
Parked For Later Async Topics

12

Async vs Sync



13

Synchronous Code Runs Now

Synchronous code runs in order:

- `A`
- `B`
- `C`

One line finishes before the next line runs.

This is the pattern most beginner code appears to follow.

14

Async Work Starts *Now* And Finishes *Later*



Asynchronous work:

- starts now
- waits for something
- finishes later
- often uses a function that runs later

The page should not feel frozen while waiting.

15

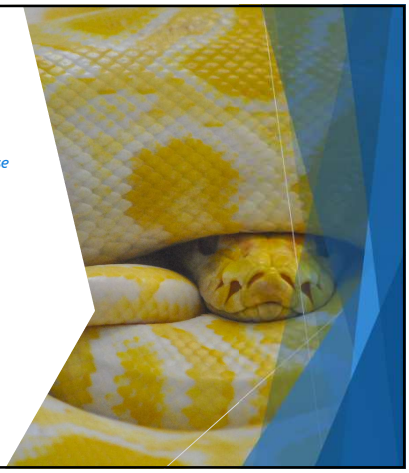
Python Bridge: Same Timing Question

Python and JavaScript use different syntax.

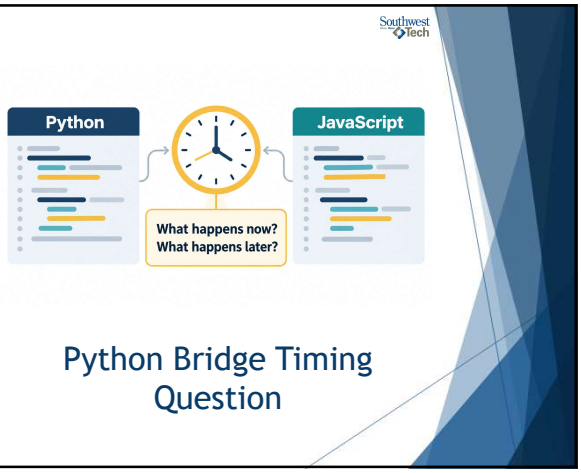
But the core question is familiar:

- What happens now?
- What happens later?
- What waits?
- What keeps going?

The timing problem is shared.



16



Python Bridge Timing Question

17

`setTimeout()` Makes Waiting Visible

`setTimeout()` says:

Run this function later.

It needs:

- a function to run later
- an amount of time to wait

The waiting is real, even when the code is short.



18

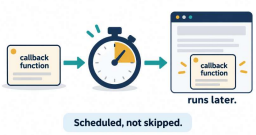
Callback: The Later Function

A callback is a function used later.

For Week 8, read it as:

"When the timer finishes, run this."

The callback is not skipped. - It is scheduled.



Scheduled, not skipped.

19

User Feedback During Waiting



Good timed behavior gives feedback:

- starting
- waiting
- finished

Example:

Click button -> "Waiting..." -> "Finished."

The user should not have to guess.

20



AI Can Explain Timing, Not Replace Your Work

AI as Explainer - You may ask:

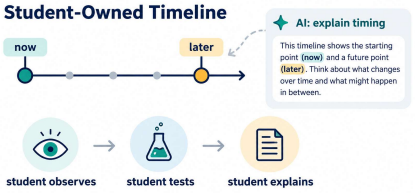
"Explain why this message appears after the other message."

You may not submit:

- AI-generated timed behavior you cannot explain
- code pasted without testing
- a reflection written from AI's point of view

You still observe, test, and explain.

21



AI Timing Explainer

22



Useful AI Prompt Pattern

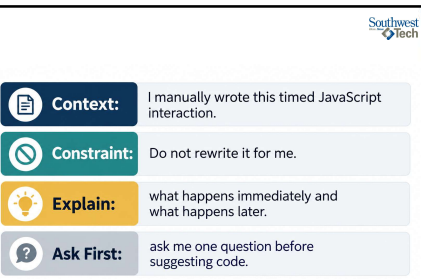
"I manually wrote this timed JavaScript interaction.

Do not rewrite it for me.

Explain what happens immediately, what happens later, and why the delayed message appears after the waiting message.

Ask me one question before suggesting code.

23



Useful AI Prompt Pattern

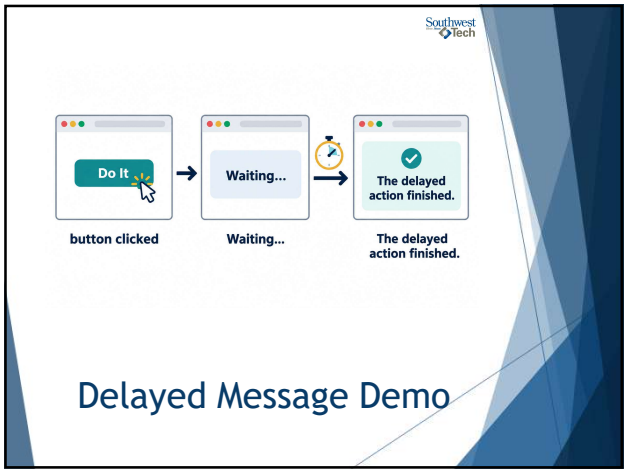
24



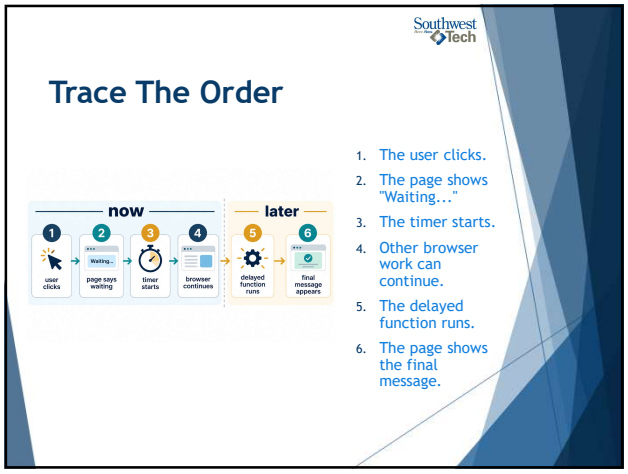
25



26



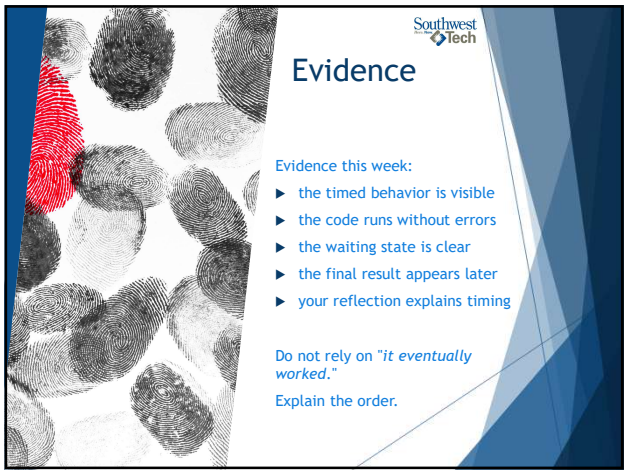
27



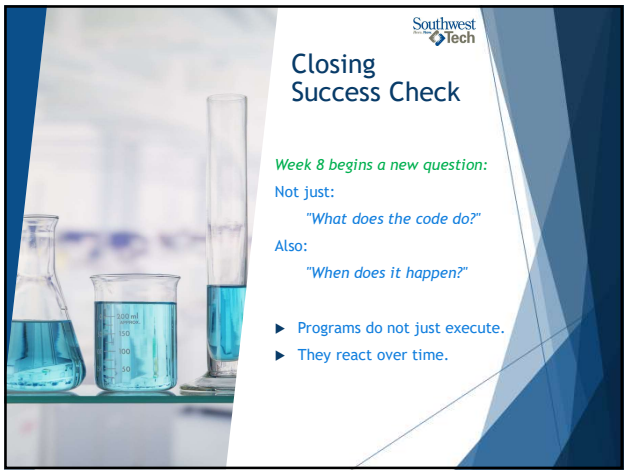
28



29



30


Southwest Tech
Closing
Success Check
Week 8 begins a new question:
Not just:
"What does the code do?"
Also:
"When does it happen?"
Programs do not just execute.
They react over time.

31


Southwest Tech
Thank you!
Have Fun!

32